

Laboratorio 1 – Primeros desarrollos

1 Objetivos

Los objetivos de esta sesión son:

- Que aprendáis a crear, compilar y ejecutar proyectos en el entorno NetBeans.
- Que aprendáis a **UTILIZAR EL DEPURADOR PARA IDENTIFICAR Y CORREGIR LOS ERRORES DE DISEÑO** en los programas que desarrolléis.

2 Qué son los Entornos de desarrollo integrado (Integrated Development Environment -IDE)

Estrictamente hablando, el desarrollo de un programa en Java puede llevarse a cabo utilizando pocas herramientas: bastarían un editor de texto simple y el entorno de desarrollo (o JDK, Java Development Kit) suministrado por los creadores de Java.

Ahora bien, cuando debe gestionarse proyectos complejos, es necesaria una herramienta más potente que facilite el trabajo al desarrollador. Tales herramientas se conocen como IDEs (“Integrated Development Environments” o Entornos de Desarrollo Integrado). Un IDE es un programa que integra en un único entorno gráfico editor, compilador, intérprete, depurador de código, así como una serie de mecanismos automatizados que facilitan el trabajo del programador, como por ejemplo, la refactorización. Existen varios IDE disponibles para Java. Para los laboratorios de la asignatura hemos elegido el entorno **Netbeans**.

En esta sesión os dedicaréis a familiarizaros con el entorno de desarrollo integrado **NetBeans** y de forma especial **CON EL DEPURADOR INTEGRADO EN EL ENTORNO**.

Debéis saber **QUE LA SINTAXIS DE JAVA ES SIMILAR A LA DE C EN LO TOCANTE A DECLARACIONES DE VARIABLES LOCALES E INSTRUCCIONES DE CONTROL DE FLUJO (CONDICIONALES, ITERACIONES, ETC.)**. En consecuencia, asumiremos que, después de cursar FO, las conocéis y sabéis usarlas correctamente.

La sección 3 de este documento muestra con cierto nivel de detalle cómo hay que usar NetBeans para crear un proyecto, compilarlo, ejecutarlo y depurarlo.

La sección 4 presenta el algoritmo de Euclides para calcular el máximo común divisor de dos números enteros) para que construyáis diversas variantes de código que lo implementen (en una versión iterativa dentro de la función main(), en una versión iterativa dentro una función, y en su versión recursiva dentro de una función).

3 Algunas prestaciones del IDE NetBeans

La primera tarea consistirá en la creación de un programa que presente por pantalla el mensaje “HelloWorld” con el entorno NetBeans. Ello os permitirá practicar en la creación de un proyecto NetBeans, la creación de una clase Java con un solo método: el método main(), su compilación y su ejecución.

3.1 Creación de un Proyecto Netbeans

Un proyecto NetBeans contiene una aplicación completa.

Cread un nuevo proyecto, yendo al menú principal de NetBeans y seleccionando las siguientes opciones: **File→New Project**, tal y como se muestra en la figura que sigue.

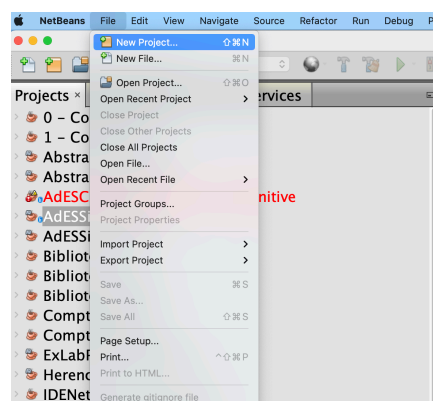


Figura 1. Creación de proyecto. Selección del tipo de proyecto.

Eso lanza el asistente de creación de proyectos. Dicho asistente presenta secuencialmente al usuario las dos pantallas que se muestran en las figuras 2 y 3. La primera solicita el tipo de proyecto que se desea crear. En esta asignatura siempre crearemos aplicaciones de Java. Para ello seleccionamos “Java with Ant” en la parte izquierda de la primera pantalla y en su parte derecha seleccionamos “Java Application”.

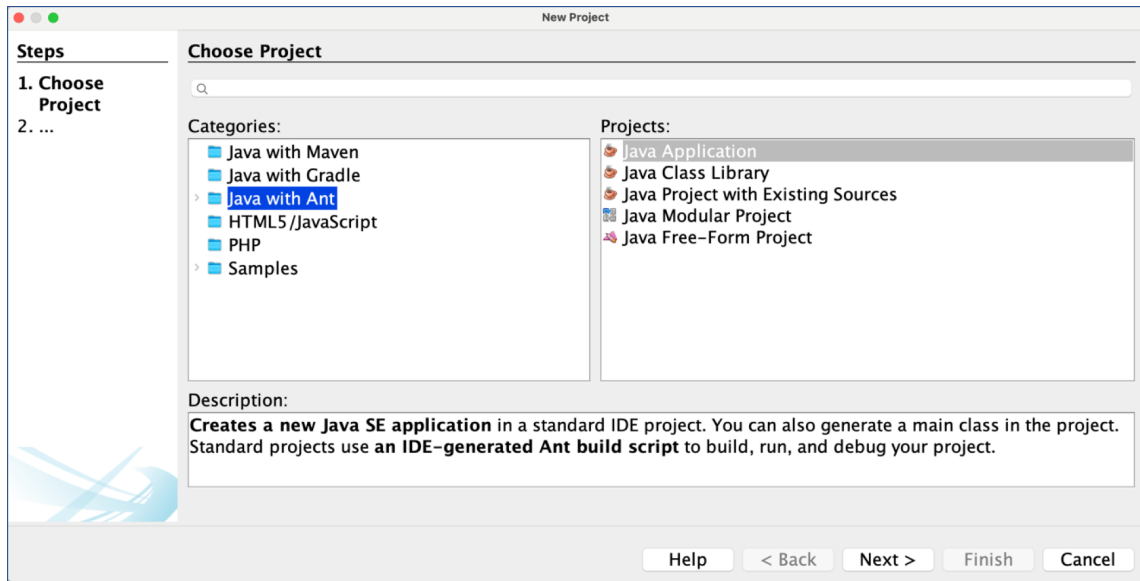


Figura 2. Creación de proyecto. Selección del tipo de proyecto.

El asistente presenta entonces la pantalla que se muestra en la figura que sigue. En ella debe introducirse la información relativa al nombre y ubicación del proyecto.

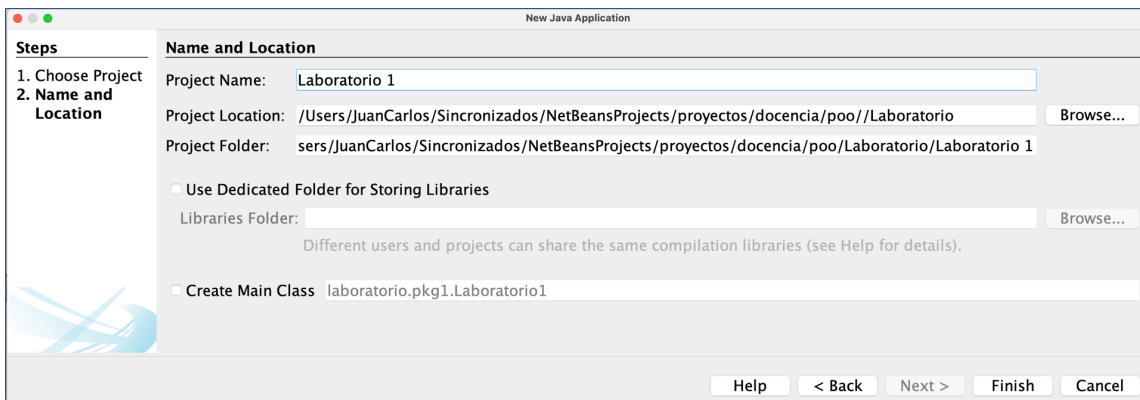


Figura 3. Creación de proyecto. Ubicación, nombre del proyecto y otros.

A continuación, se muestran los principales elementos de esta pantalla.

- Nombre del proyecto (Project Name): Es el identificador que se asignará al proyecto dentro del entorno NetBeans. Para este proyecto seleccionad “Laboratorio 1”.
- Ubicación del proyecto (Project Location): Carpeta del sistema de archivos del sistema en el que NetBeans creará las carpetas y archivos necesarios para gestionar el proyecto. Podéis utilizar cualquier carpeta. **DEBERÉIS TENER EN CUENTA QUE SI ELEGÍS UNA CARPETA LOCAL, DEBERÉIS SALVAR SU CONTENIDO EN VUESTRAS ZONAS O PERDERÉIS EL TRABAJO.** En consecuencia, se sugiere que elijáis una carpeta en vuestras zonas de trabajo.
- Crear clase principal (Create Main Class): Si esta opción está activa, NetBeans genera automáticamente una clase llamada Main. **Se sugiere desactivar esta opción para que podáis crear la clase que incluya el método main() que deseáis.**

Cuando el asistente finaliza su trabajo, el llamado explorador de proyectos de NetBeans, que aparece en la parte superior izquierda de la pantalla, presenta el nuevo proyecto tal y como se muestra en la figura que sigue.

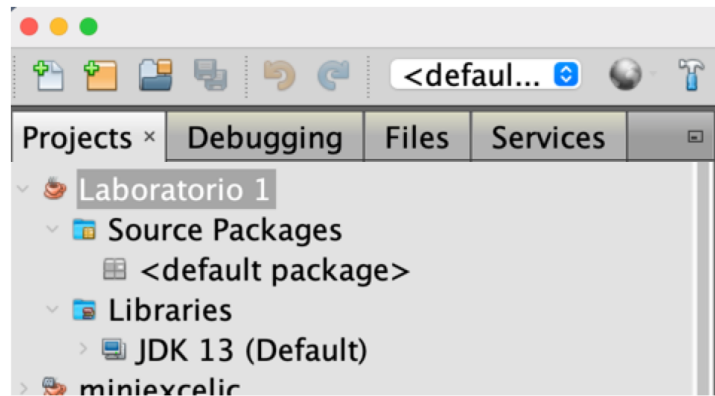


Figura 4. Creación de proyecto. Proyecto creado mostrado en Explorador de proyectos de NetBeans.

3.2 Creación de packages

El código Java de un proyecto se distribuye en clases que se agrupan en lo que se conoce como **packages**. Cualitativamente un package es un grupo de clases lógicamente relacionadas (por ejemplo, en Java todas las clases que gestionan operaciones de Entrada/Salida basadas en flujos de datos –streams– se agrupan en el package `java.io` y las clases que implementan los diferentes tipos de contenedores se agrupan en el package `java.util`). **Toda clase de Java pertenece a un package.**

A cada package se le asigna un nombre consistente en una secuencia de strings separados por el carácter '.' (java.util, java.io, java-awt-image.renderable, etc.). En un proyecto NO puede haber dos packages con nombres idénticos. Para evitar colisiones de nombres de packages las entidades en las que se desarrolla software Java asignan a éstos nombres relacionados con sus respectivos dominios en Internet, puesto que dichos dominios son únicos en todo el mundo. En Java es común asignar a los packages los dominios invertidos sin "www".

Así por ejemplo si el dominio de la ETSETB es www.etsetb.upc.edu, los nombres de los packages desarrollados para la realización del código de la asignatura POO podrían comenzar con **"edu.upc.etsetb.poo"** y seguir con componentes que identificasen el contenido de cada package desde un punto de vista lógico.

Desde un punto de vista físico, un package en un programa Java implica la existencia en un cierto punto del sistema de archivos del computador de un SUBÁRBOL DE CARPETAS CUYOS NOMBRES SON PRECISAMENTE LOS COMPONENTES DEL NOMBRE DEL PACKAGE EN CUESTIÓN.

En este paso crearéis un package para depositar vuestras clases y observaréis el subárbol de carpetas que la creación de dicho package ha generado.

Cuando se crea un proyecto, NetBeans genera lo que se conoce como un package por defecto (default package). Se recomienda encarecidamente NO TRABAJAR NUNCA CON EL PACKAGE POR DEFECTO.

Cread un nuevo package para el proyecto seleccionando el proyecto en el explorador de proyectos, pulsando el botón derecho del ratón y seleccionando New→Java Package, tal y como se muestra en la figura que sigue.

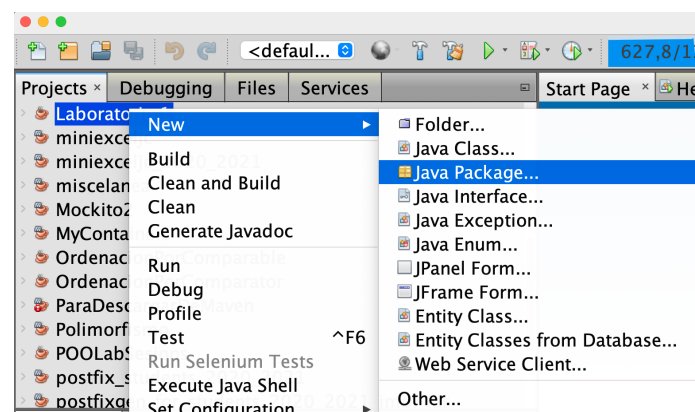


Figura 5. Creación de package. Solicitando crear un nuevo package.

NetBeans muestra entonces la pantalla que se muestra a continuación.

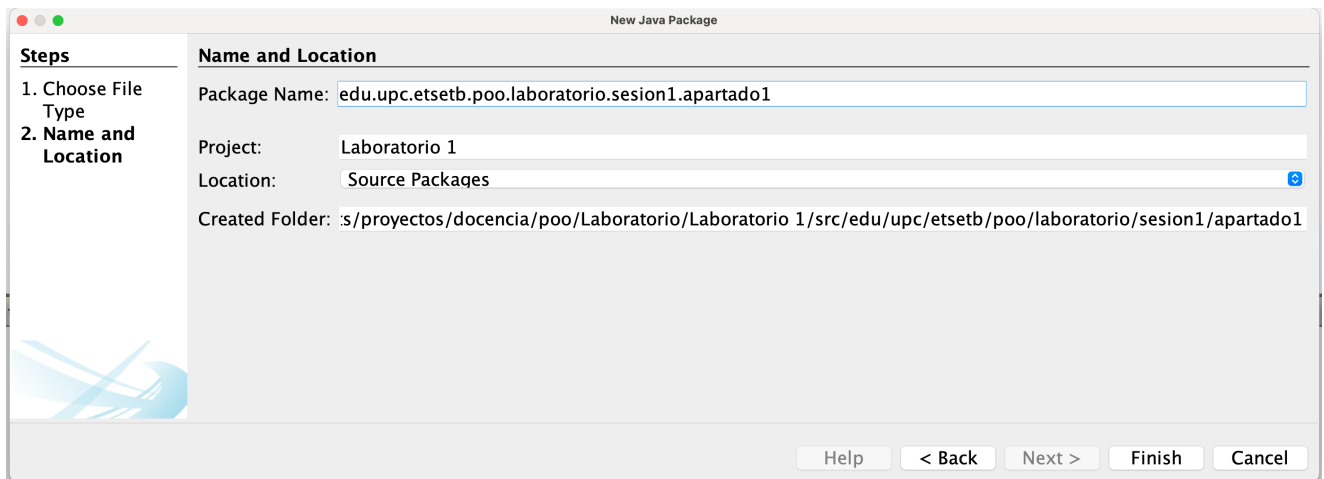


Figura 6. Creación de package. Asistente para seleccionar el nombre del package.

En ella debéis introducir el nombre del package. Se sugiere que asignéis al package el nombre **“edu.upc.etsetb.poo.laboratorio.sesion1.apartado1”**.

Una vez hecho esto, el explorador de proyectos muestra en el nuevo package dentro del proyecto.

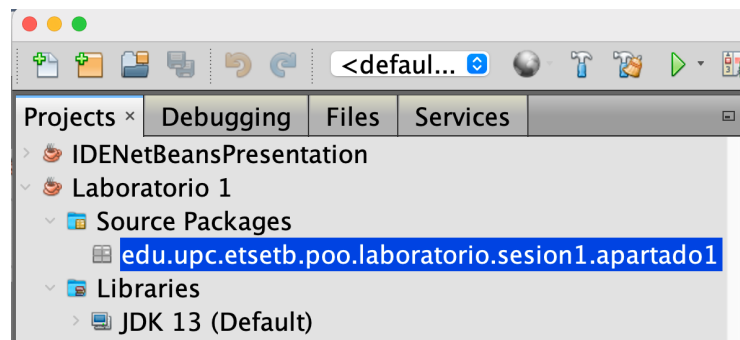


Figura 7. Creación de package. Package creado mostrado en el Explorador de proyectos.

Abrid en vuestro computador un explorador del sistema de archivos. Id a la carpeta donde habéis creado el proyecto. Allí encontraréis la carpeta “src”. NetBeans deposita el código fuente que generéis dentro de esta carpeta, a no ser que le indiquéis lo contrario. Observad ahora el contenido de la carpeta: en ella encontraréis una carpeta **“edu”**, y dentro de ella la carpeta **“upc”**, y así sucesivamente hasta llegar a la subcarpeta **“apartado1”**: la creación de un package ha conllevado la creación de un subárbol de carpetas cuyos nombres coinciden con los componentes del nombre del package.

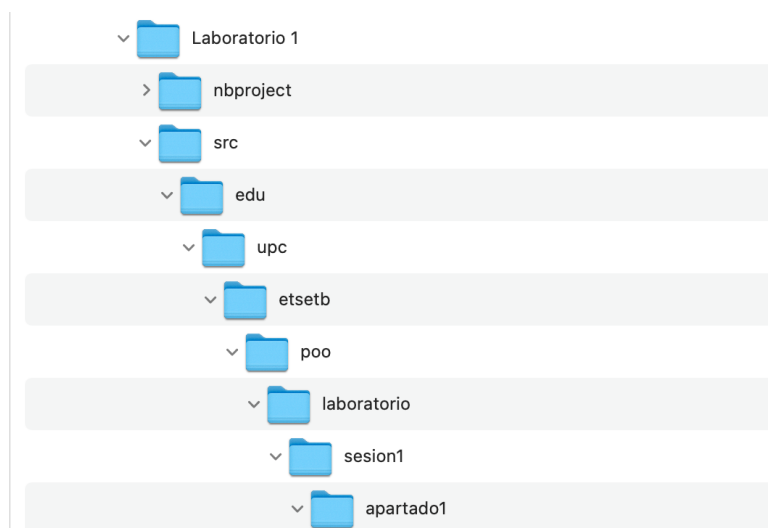


Figura 8. Creación de package. Carpetas correspondientes al package creado.

3.3 Creación de clases

Una vez se ha creado un package, hay que generar las clases que lo conforman. En nuestro caso, el package creado solo contendrá la clase HelloWorld.

Cread una nueva clase para el package seleccionando. Para ello, seleccionad EL PACKAGE RECIÉN CREADO en el explorador de proyectos, pulsad el botón derecho del ratón y seleccionad *New→Java Class* tal y como se muestra en la figura que sigue:

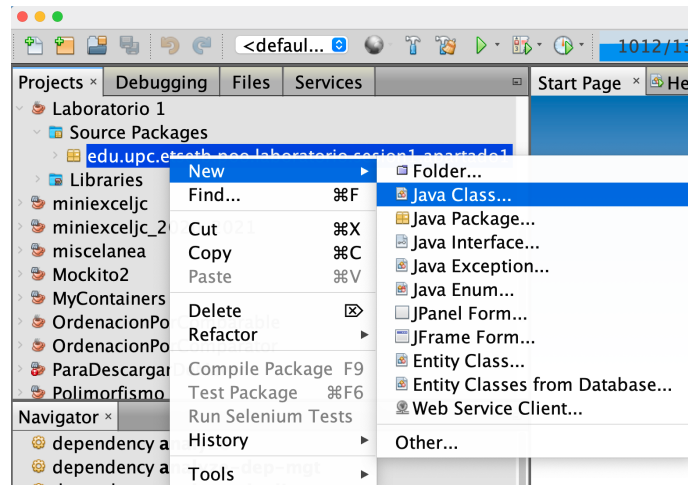


Figura 9. Creación de una clase. Solicitando la creación de una clase.

NetBeans muestra, a continuación el asistente mostrado en la figura que sigue.

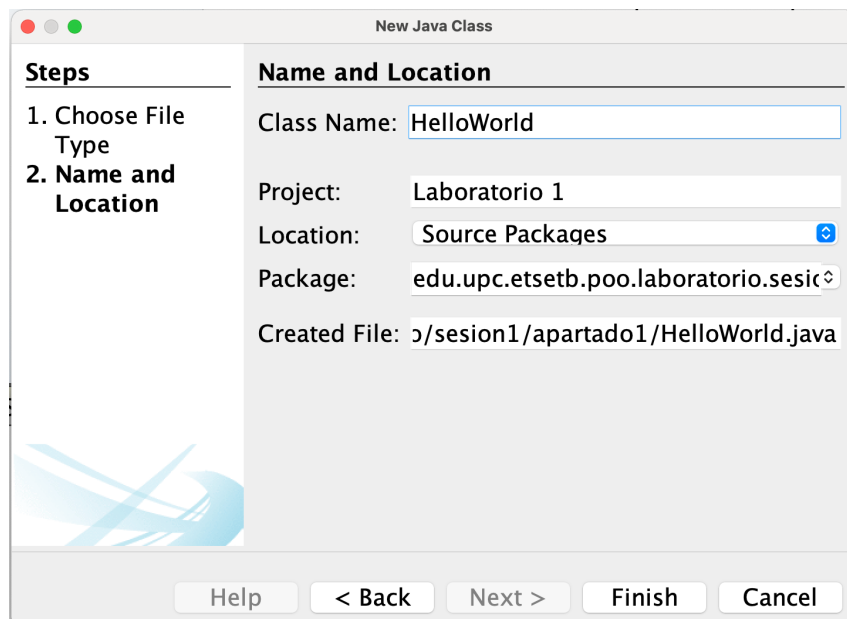


Figura 10. Creación de una clase. Asistente para seleccionar nombre de la clase.

Cumplimentad el formulario asignando a la clase el nombre HelloWorld. NetBeans crea entonces una clase vacía muestra su contenido en la ventana de edición.

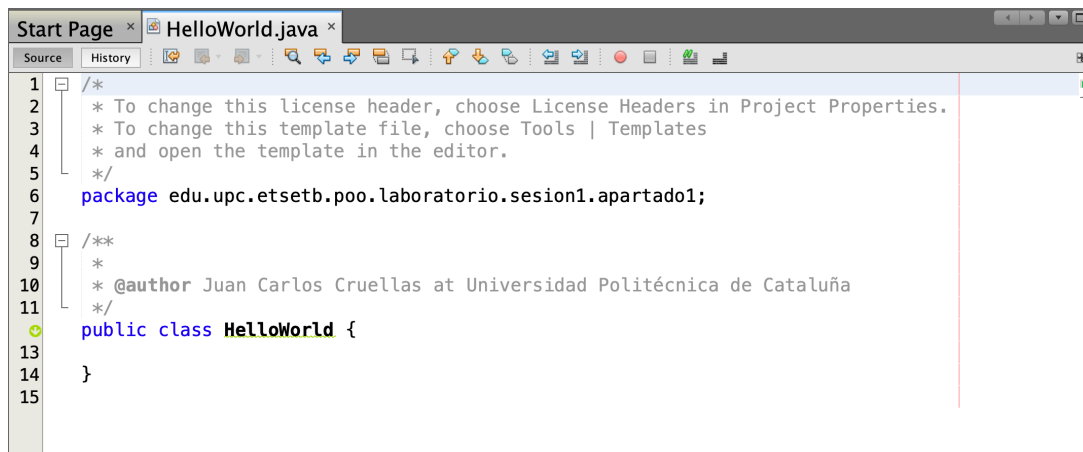


Figura 11. Creaci n de una clase. Clase vac a creada por el asistente de NetBeans.

Adem s, explorador de proyectos de proyectos de NetBeans muestra la clase dentro del package en el que se ha creado.

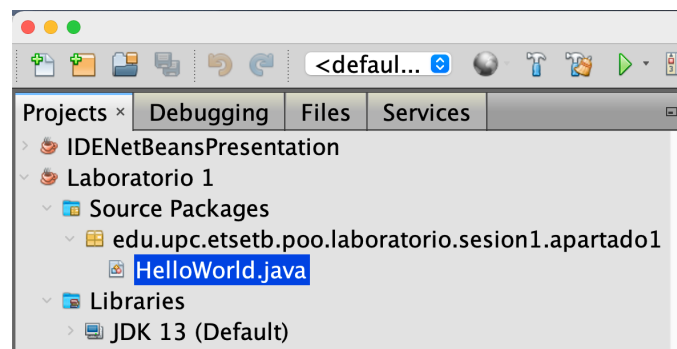


Figura 12. Creaci n de una clase. Clase reci n creada mostrada en el explorador de proyectos dentro de su package.

Al mismo tiempo, NetBeans genera en el sistema de archivos del computador el archivo que contiene a la clase (HelloWorld.java) dentro de la subcarpeta correspondiente, seg n el nombre del package al que la clase pertenece. Abrid en vuestro computador un explorador del sistema de archivos. Id a la carpeta donde hab is creado el proyecto. All  encontrar is la carpeta "src/edu/upc/etsetb/poo/laboratorio/sesion1/apartado1". Observad su contenido: NetBeans ha creado un archivo HelloWorld.java. Este ser  el archivo en el que NetBeans depositar  el c digo fuente de la clase HelloWorld.

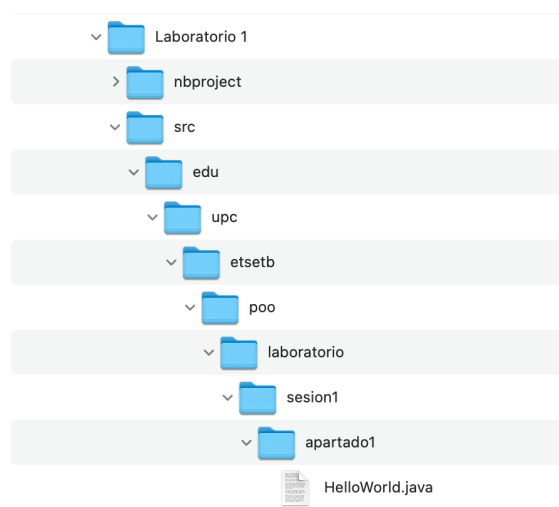


Figura 13. Creaci n de una clase. Archivo conteniendo la clase reci n creada dentro de la carpeta correspondiente seg n el nombre del package.

3.4 Edici n de las clases

Una vez creada la clase hay que a adir c digo fuente. Cuando se crea una nueva clase NetBeans abre el editor en la pantalla principal (ventana derecha superior; suele ser la mayor de todas), tal y como se ha mostrado anteriormente. Mediante el explorador de proyectos pod is abrir en el editor cualquier clase haciendo un doble

click sobre la clase en cuestión.

Editad ahora la clase “HelloWorld” para que su código sea el indicado a continuación. Podéis intentar copiar y pegar el texto directamente desde este PDF, pero debéis tener en cuenta que en ocasiones al hacerlo se insertan caracteres de control invisibles que pueden ocasionar errores de compilación. Si eso ocurriera deberíais eliminarlos o bien escribir el código en lugar de copiarlo y pegarlo.

```
package edu.upc.etsetb.poo.laboratorio.sesion1.apartado1;
```

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello world!");  
    }  
}
```

La ejecución de la sentencia `System.out.println("Hello World");` hará que el computador presente en la consola de ejecución el mensaje “Hello World” seguido de un final de línea. Lo que se muestra encerrado entre paréntesis es el valor de un `String` en Java. Si en lugar de dicho `String` se hubiera insertado el nombre de una variable de tipo `int`, el programa hubiera mostrado en la consola la representación textual del valor entero de dicha variable. Lo mismo sucede si se pasa como argumento una variable de tipo primitivo (real, double, boolean, etc.). A lo largo del cuatrimestre haréis un uso intensivo de esta sentencia.

La sentencia `System.out.print(...)` hace exactamente lo mismo que la anterior salvo que NO incluye el salto de línea al final.

3.5 Compilación

Ningún computador es capaz de entender y ejecutar el código fuente de un archivo Java. Antes de ejecutar es necesario compilar el código fuente. De ello se encarga el compilador de Java. El compilador de Java traduce el código fuente existente en todos los archivos fuente del proyecto a un código binario denominado **bytecode**. **Cada archivo .java del proyecto da lugar a un archivo bytecode.** El nombre del archivo bytecode coincide con el del archivo del código fuente. Su sufijo es **.class**. Así el compilador genera el archivo bytecode **HelloWorld.class** después de compilar el archivo **HelloWorld.java**.

El compilador de NetBeans compila conforme el programador edita el contenido del archivo. Aún así, el programador puede solicitar la compilación de un determinado archivo seleccionando el archivo desde el explorador de proyectos, pulsando el botón derecho del ratón y seleccionando la opción “Compile File” del menú contextual presentado por NetBeans.

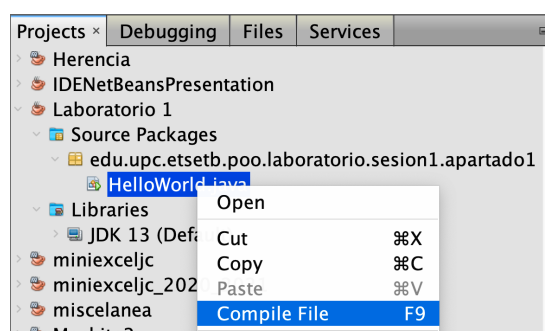


Figura 14. Compilación de una clase. Solicitando explícitamente la compilación de una cierta clase.

El proceso de compilación mantiene las clases en los packages que se les asignaron en tiempo de edición. Así pues, los archivos `.class` se distribuyen en subárboles de carpetas idénticos a los subárboles en los que se distribuyen los archivos `.java`. En concreto, al compilar, NetBeans genera una carpeta `build`; dentro de ella crea la subcarpeta “`classes`”, y dentro de ésta genera los subárboles de carpetas cuyos nombres se corresponden con los componentes de los diferentes packages existentes en el proyecto.

Observad en la figura que sigue el paralelismo entre los árboles de carpetas presentes en “`classes`” y “`src`”.

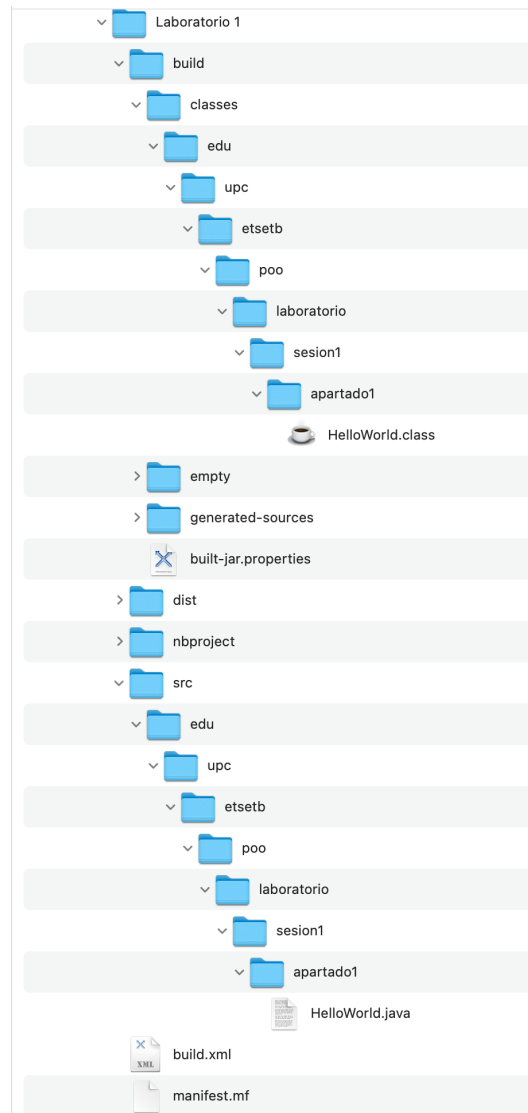


Figura 15. Compilación de una clase. Árboles de directorios para código fuente y para bytecode (clases compiladas).

NetBeans puede configurarse para que compile el código fuente mientras editamos dicho código fuente. Es por ello que en estos casos, mientras estamos editando el compilador nos va indicando errores de compilación, subrayando aquellas sentencias que son erróneas. Eso hace que mientras no terminemos una determinada sentencia, NetBeans nos la subraye indicando error de compilación. Resulta conveniente tener habilitada esta opción para despreocuparnos de ordenar al NetBeans que compile cuando modificamos uno o varios archivos de un proyecto grande.

Para habilitar la opción de compilación automática hay que ir al explorador de proyectos, pulsar el botón derecho del ratón sobre el proyecto y seleccionar Properties, tal y como se muestra en la figura que sigue.

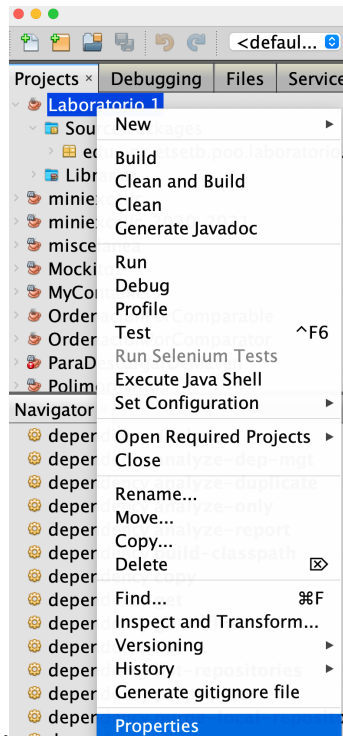


Figura 16. Perfilando la compilación. Primer paso para solicitar compilación al salvar: ir a ventana de propiedades del proyecto

NetBeans presentará un asistente en el que podremos definir determinadas propiedades del proyecto. Seleccionad Compile. Una vez hecho esto, NetBeans mostrará lo siguiente.

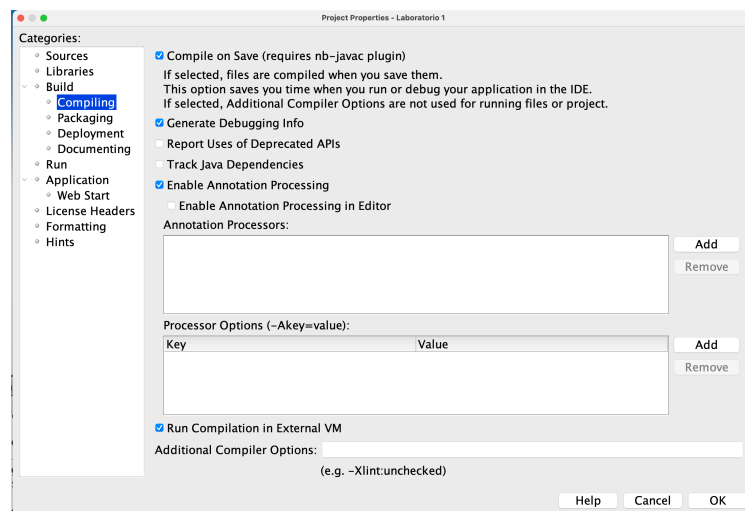


Figura 17. Perfilando la compilación. Opción "Compile on Save" activada.

Aseguraos de que la opción "Compile on Save" esté activada. Eso hará que el compilador compile una clase cada vez que salvéis el archivo que la contiene.

En caso de no tener habilitada esta opción, hay dos formas de ordenar a NetBeans la compilación de una cierta clase:

1. Seleccionad la clase a compilar en el explorador de proyectos, pulsad el botón derecho del ratón y seleccionad "Compile File" como se muestra en la figura que sigue.

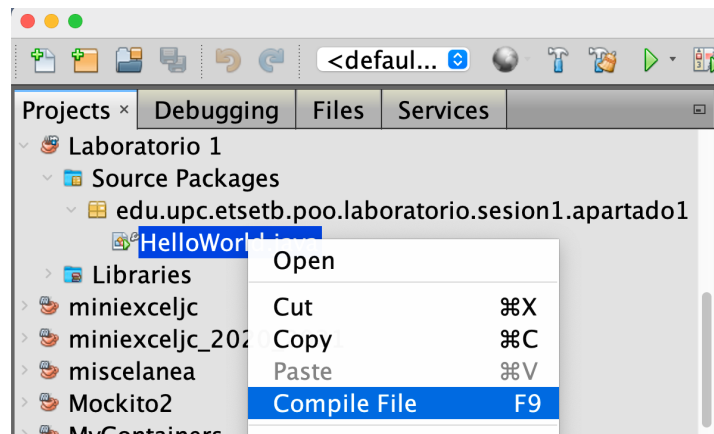


Figura 18. Compilando cuando la opción "Compile on Save" NO está activada. A través del explorador de proyectos.

2. Haced que NetBeans os muestre la clase a compilar en la ventana de edición. Pulsad entonces la tecla F9.

Os recomendamos mantener la opción "Compile on Save" activada.

Si en el explorador de proyectos aparece una exclamación al lado del icono representativo de la clase, el código de dicha clase contiene errores de compilación que hay que eliminar, puesto que de lo contrario el programa no podrá ejecutarse. Como se ha comentado, el editor muestra los errores subrayando en rojo las líneas que los contienen. Si detenemos el puntero del ratón sobre ellas unos segundos, NetBeans nos muestra un recuadro con una explicación del error de compilación. ES IMPORTANTE LEER ATENTAMENTE LA INFORMACIÓN QUE NETBEANS MUESTRA SOBRE CADA ERROR DE COMPILACIÓN PUESTO QUE SOLO ENTENDIENDO CÚAL ES EL ERROR Y SUS CAUSAS, PODREMOS SUBSANARLO.

Para probar este mecanismo eliminad el ';' del final de la línea con el código "System.out.println(...)"; si tenéis habilitada la opción de compilación al salvar, el editor subrayará en rojo dicha línea. Apuntad con el ratón a esa línea y esperad a que NetBeans os presente la causa del error en un recuadro tal y como se muestra en la figura que sigue. En esta figura se observa como el compilador nos dice que falta el carácter ';' al final de la línea.

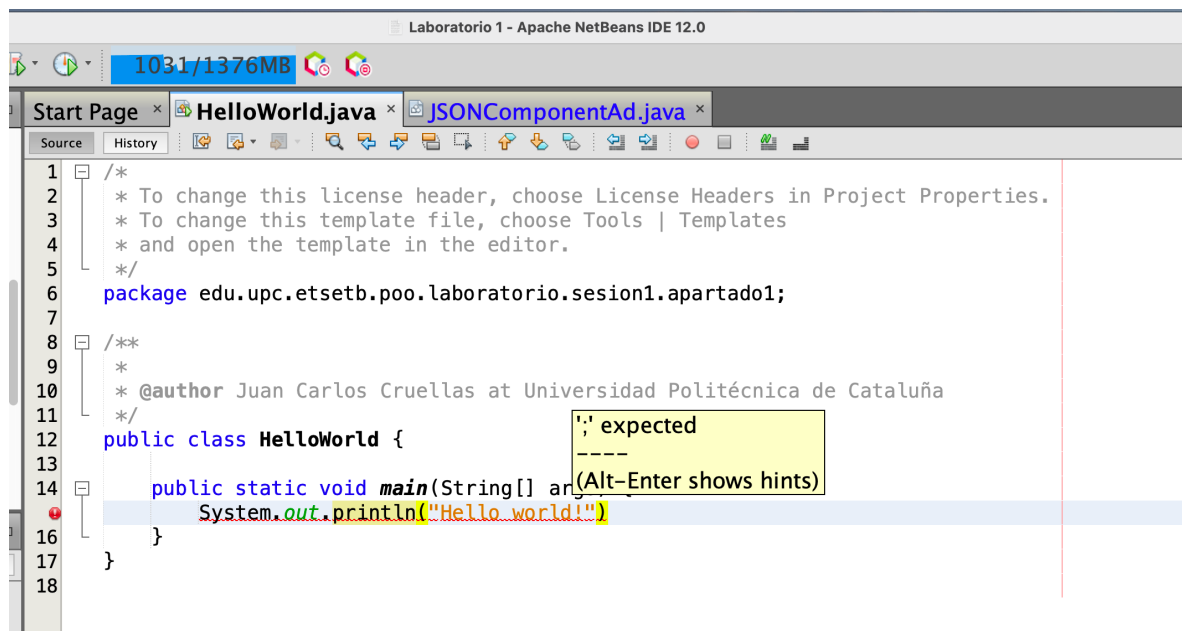


Figura 19. Detección de errores de compilación.

Cuando el código no contiene errores de compilación, podemos intentar ejecutar el programa. ES IMPORTANTE QUE TENGÁIS SIEMPRE PRESENTE QUE UN PROGRAMA SIN ERRORES DE COMPILACIÓN NO TIENE POR QUÉ FUNCIONAR BIEN NECESARIAMENTE. PUEDE CONTENER ERRORES DE DISEÑO QUE HAGAN QUE EL PROGRAMA NO FUNCIONE CORRECTAMENTE.

NOTA 1: en un proyecto Java pueden haber **varias clases con el método main()**, lo cual permite diversos puntos de comienzo de ejecución del mismo.

NOTA 2: La opción **Clean and Build** borra todos los archivos .class y compila todos los .java de nuevo. Esta opción resulta útil cuando no se está seguro de lo actualizados que están los archivos .class respecto a los .java.

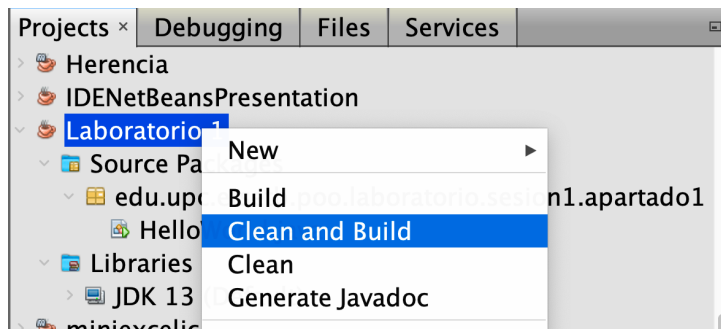


Figura 20. Compilación. Compilando de nuevo todas las clases del programa.

3.6 Ejecución

En los proyectos creados como “Java Application”, la ejecución debe dar comienzo en un método `main()` de una cierta clase. Llamaremos clase principal a aquella que contiene el método `main()`.

En el navegador de proyecto las clases que contienen el método `main()` de comienzo de ejecución aparecen marcadas con un triángulo verde idéntico al triángulo de las teclas de reproducción (play) de reproductores de CDs, vídeos, etc., tal y como se muestra en la figura anterior (notad el triángulo en `HelloWorld`).

Existen varias formas de lanzar la ejecución de un método `main()`.

1. Abrid en el editor la clase que contiene el método `main()`, pulsad el botón derecho del ratón y seleccionad la opción “Run File”. Si pulsáis simultáneamente las teclas `Shift+F6`, obtendréis el mismo resultado.

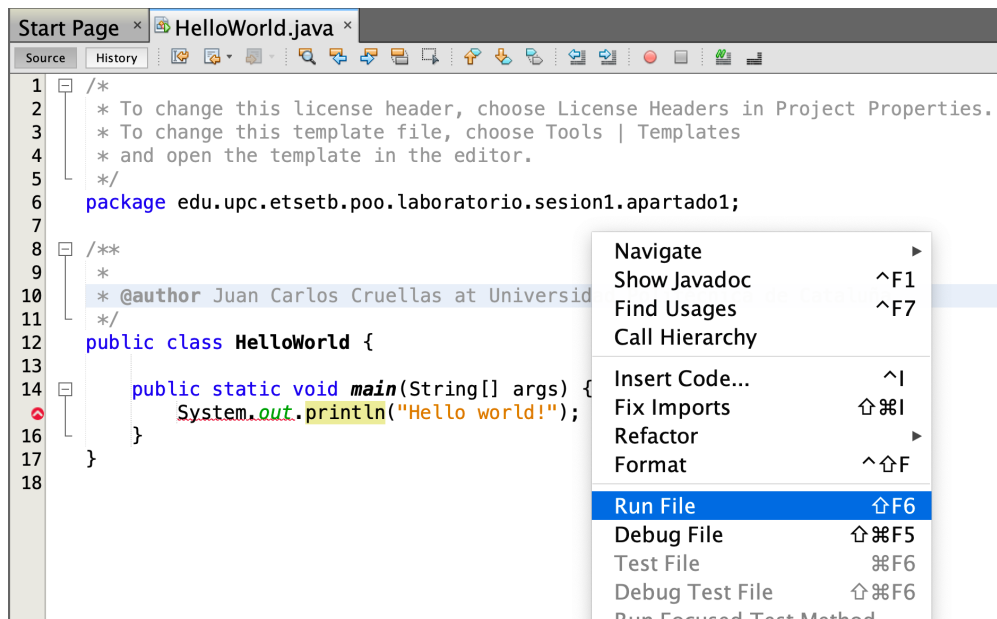


Figura 21. Lanzando la ejecución. Desde la ventana de edición.

2. Seleccionad el archivo que contiene la clase con el método `main()` en el explorador de proyectos, pulsad el botón derecho del ratón y seleccionad “Run File” tal y como se muestra en la figura que sigue.

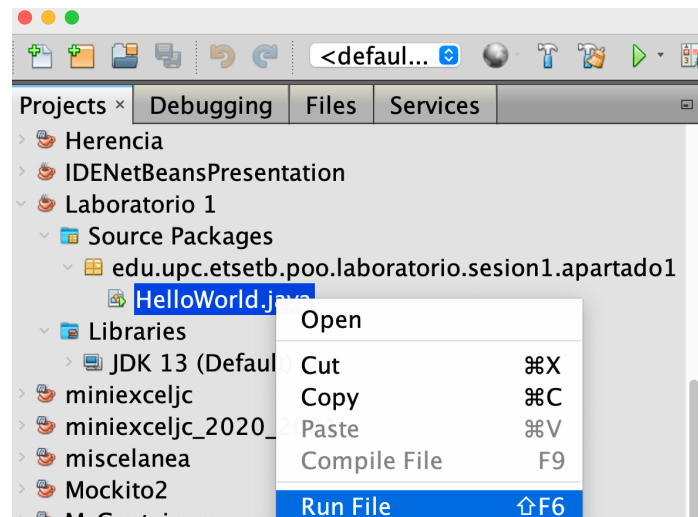


Figura 22. Lanzando la ejecución. Desde el explorador de proyectos.

Al ejecutar un programa NetBeans abre la ventana correspondiente a la consola (normalmente en la parte inferior derecha) y muestra los mensajes que la ejecución haya ordenado presentar en dicha consola. La salida del programa HelloWorld debería dar lugar a una salida como la mostrada en la figura que sigue.

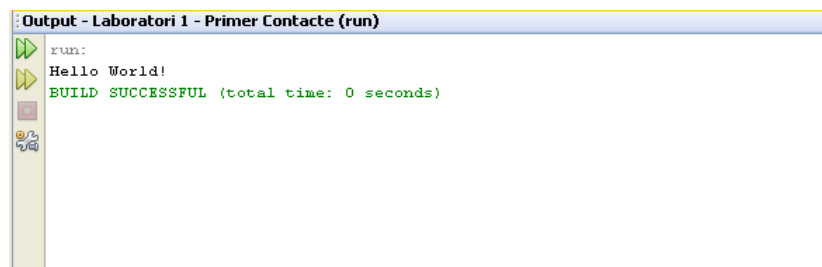


Figura 23. Mostrando resultado de ejecución en la consola de NetBeans.

Es conveniente configurar NetBeans para que por defecto nos muestre la ventana correspondiente a la consola. Para ello basta con seleccionar en la barra superior de opciones Window→Output.

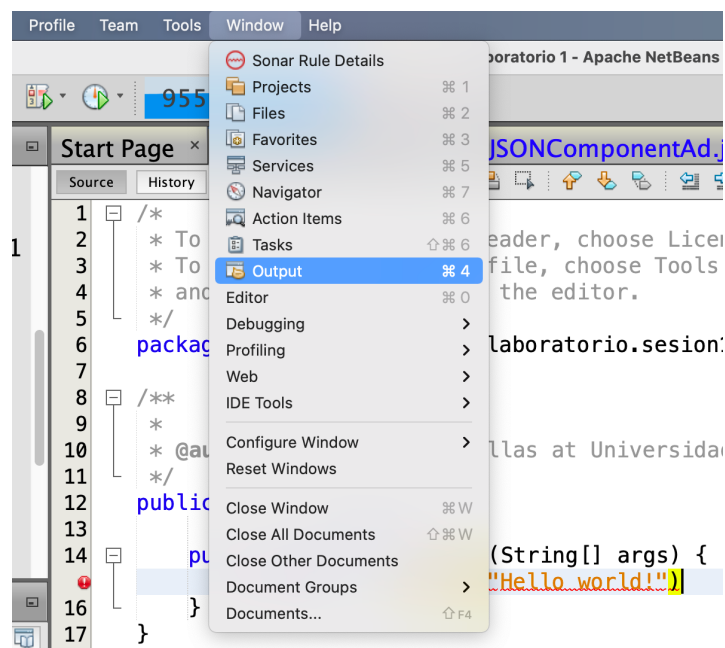


Figura 24. Ordenando mostrar la ventana de consola (Output) en la parte inferior derecha del entorno NetBeans.

3.7 Introducción al depurador

El depurador es un programa integrado en NetBeans que permite ejecutar programas de forma controlada. Ello permite, por ejemplo, inspeccionar los valores de las variables determinados puntos, y detectar aquellos en los que dichos valores son diferentes a los que el diseñador había previsto, lo que es indicativo de la existencia de errores de diseño.

Para lanzar la ejecución del depurador hay varias alternativas:

- Apuntad con el ratón en una de las clases que contienen el método main, pulsad el botón derecho y seleccionad la opción “Debug File”.

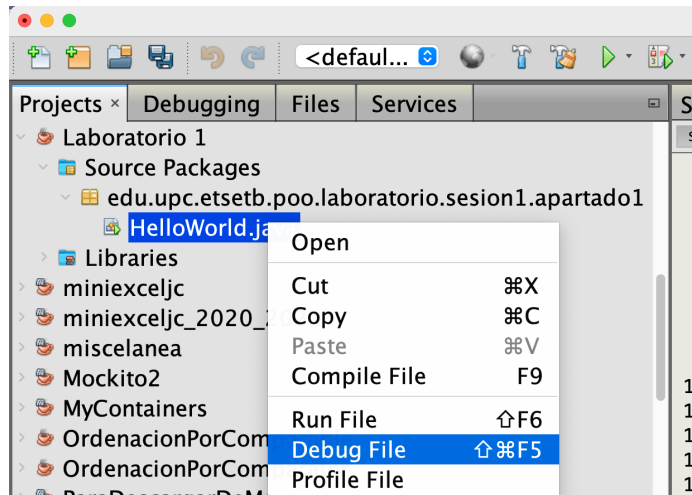


Figura 25. Depuración. Lanzando una sesión de depuración desde el explorador de proyectos

- Abrid en el editor el código de una de las clases que contienen el método main, apuntad con el ratón a la ventana de edición de dicha clase, pulsad el botón derecho y seleccionad la opción “Debug File”

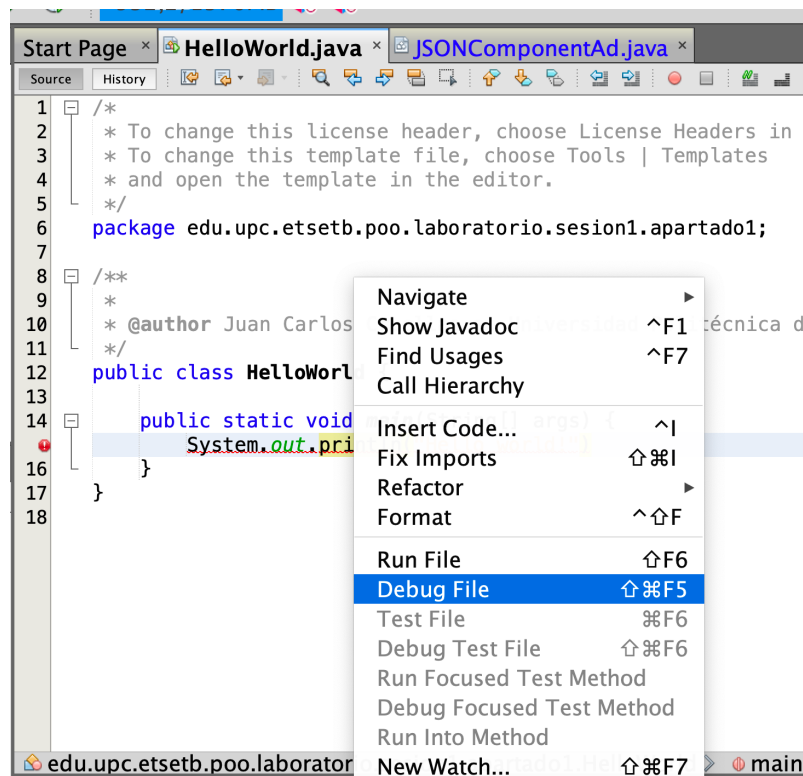


Figura 26. Depuración. Lanzando una sesión de depuración desde la ventana de edición

Una ejecución normal y una de depuración no se diferenciarán si previamente al comienzo de la sesión de depuración no se han definido puntos de parada temporal de la ejecución del programa.

El depurador de NetBeans permite, entre otras, realizar las siguientes operaciones:

1. Definir puntos de parada temporal de la ejecución del programa (breakpoints).

Para definir un punto de parada debe hacerse un doble click con el ratón a la barra franja gris de la izquierda de la ventana de edición en la línea en la que queréis que se detenga el programa. Cuando aparezca toda la línea sombreada en rojo el punto de parada habrá sido definido. En la siguiente figura se muestra el aspecto de la ventana de edición después de haber definido un punto de parada en la línea 15. Usaréis el depurador para llevar a cabo las tareas descritas en apartados posteriores.

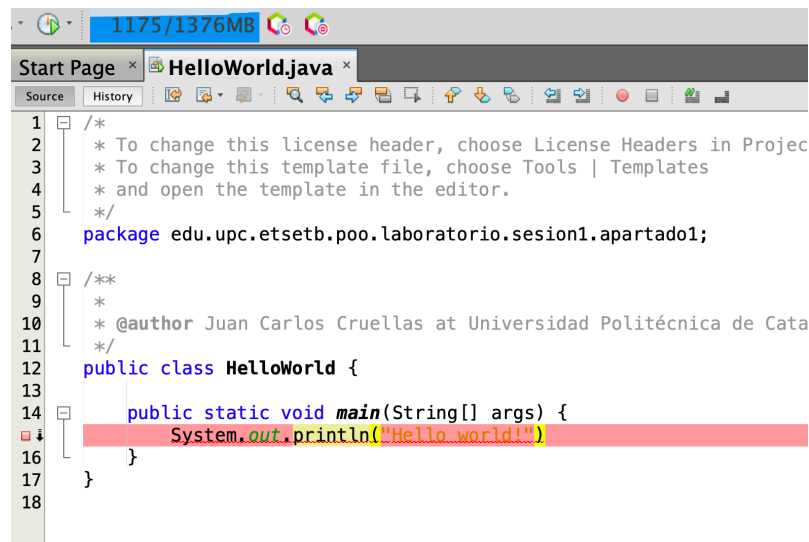


Figura 27. Depuración. Definiendo un punto de parada temporal de ejecución (breakpoint)

Otra forma de definir un punto de parada es seleccionar una línea con el ratón, pulsar botón derecho y seleccionar la opción "Toggle Line Breakpoint".

Debe tenerse en cuenta que los puntos de parada no pueden ponerse en cualquier línea del código. Es necesario crearlos sobre líneas que lleven a cabo alguna operación. No pueden ponerse en líneas en blanco, o líneas que solo declaran variables sin asignarles valores iniciales.

2. Mostrar los valores de todas las variables definidas en el ámbito de la instrucción en la que se ha detenido la ejecución temporalmente.

Para ello es necesario que antes de comenzar la sesión de depuración hayáis hecho que NetBeans os muestre la ventana "Variables" debajo del editor, seleccionando Window->Debugging->Variables en la barra superior de opciones, tal y como se muestra en la figura que sigue.

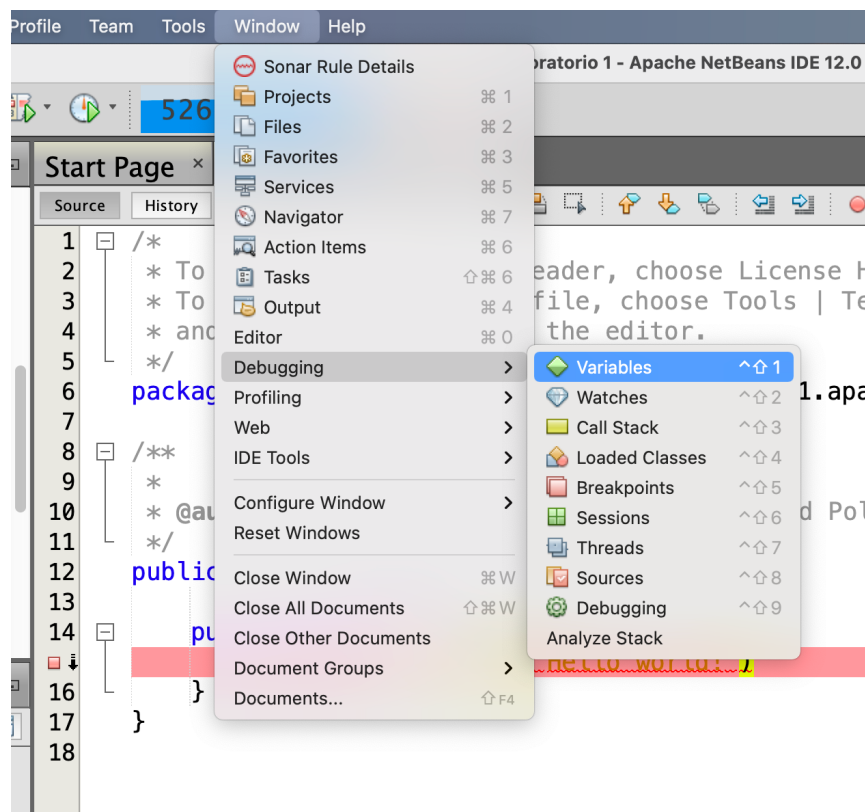


Figura 28. Depuración. Ordenando mostrar la ventana de inspección de valores de variables en la parte inferior derecha del entorno

3. Mostrar de forma permanente los valores de determinadas variables (watches).

Para ello hay que seleccionar Window->Debugging->Watches en la barra superior de opciones, que aparece justo debajo de la opción Variables, tal y como puede observarse en la figura anterior.

4. Ejecutar el conjunto de instrucciones de la línea en la que se ha detenido la ejecución del

programa (Step Over: F8 o icono mostrado en la Figura 29).

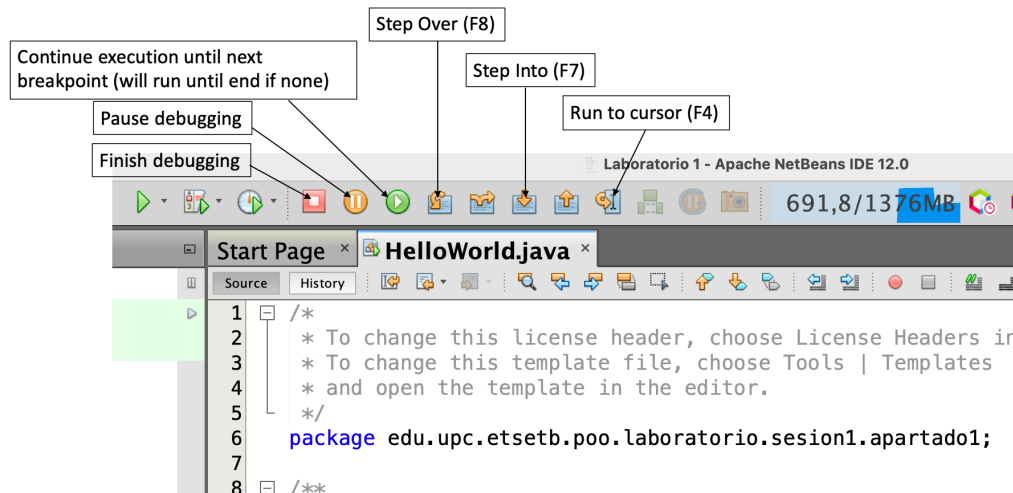


Figura 29. Depuración. Controlando la ejecución

5. Comenzar la ejecución de una función invocada en la línea en la que se ha detenido la ejecución del programa y detenerse en la primera instrucción de dicha función (Step Into: F7 o icono mostrado en la Figura 29)
6. Continuar la ejecución desde el punto de parada en el que se ha detenido el programa hasta el siguiente punto de parada definido (F5 o icono indicado en la Figura 29).
7. Acabar la sesión de depuración (Shift + F5 o icono de finalización indicado en la Figura 29).

Cuando el programa detiene temporalmente su ejecución en el primer breakpoint en la ventana Variables se muestran todas las variables del ámbito en el que se encuentra la instrucción en la que se ha detenido la ejecución. Si ahora se ejecutan las instrucciones de la línea de parada (Step Over, F8), el cursor de ejecución avanzará a la siguiente línea y la ventana Variables permitirá inspeccionar los nuevos valores de las variables.

De esa forma, en caso de que el programa no funcione correctamente, podremos ir viendo cómo las distintas variables van cambiando sus valores hasta que detectemos que alguna variable toma un valor no esperado: en ese momento estaremos más cerca de determinar las causas del mal funcionamiento del programa.

4 Implementación y depuración del algoritmo de Euclides

En esta sesión implementaremos el algoritmo de Euclides para calcular el máximo común divisor de dos números enteros m y n .

A continuación, os mostramos una descripción de dicho algoritmo:

Hacer que el dividendo sea el mayor de m y n y que el divisor sea el menor de ellos.

Calcular el resto de la división entera.

Mientras el resto de la división entera sea distinto de cero

 Hacer que el dividendo sea ahora el anterior divisor

 Hacer que el divisor sea ahora el resto de la división entera

 Calcular el nuevo resto de la división entera

Al acabar el bucle, el resultado será el resto de la penúltima división entera calculada.

El código que desarrolléis debe gestionar las siguientes variables:

- m y n serán los números cuyo mcd desea calcularse.
- Una variable para guardar el dividendo, otra para guardar el divisor y otra para guardar el resto de la división entera.

Por último, observad que el resultado se ha guardado en la variable que guardaba el divisor.

4.1 Implementación en el main()

Cread un nuevo proyecto NetBeans y llamadlo "Laboratorio1–Euclides".

Cread en él el package "edu.upc.etsetb.poo.laboratorio.sesion1.apartado2".

Cread en él la clase **MaxComDivisorMainIterativo**.

Añadid dentro de esa clase el método `main()`. Implementad dentro de ese método el algoritmo de Euclides.

NOTA IMPORTANTE: Los valores de `m` y de `n` NO se preguntan al usuario. En su lugar se les asigna valor utilizando sentencias de asignación, como se muestra a continuación

```
int m = 8, n=16;
```

Esos valores deberán cambiar una vez el código arroje el resultado correcto para ese par de valores. Eso permite comprobar que el código funcione con otros pares.

Antes de proceder a generar código, contestad a las siguientes preguntas y tened en cuenta las respuestas a la hora de generar el código:

1. ¿Cuándo NO se ejecutará el bucle?
2. ¿Qué valor deberá tomar entonces el resultado?

Cuando hayáis acabado de implementar el código, preparad en un papel los valores que las diferentes variables deben tomar en cada iteración y UTILIZAD EL DEPURADOR PARA SEGUIR LA EJECUCIÓN PASO A PASO Y COMPROBAR SI LOS VALORES QUE TOMAN SON LOS QUE PENSABÁIS. En caso de que no sea así identificad los puntos en que hay discrepancias, averiguad por qué sucede eso y modificad el código para subsanar el problema.

Una vez el código funcione correctamente, modificadlo para que `m` y `n` tomen los valores mostrados en la tabla y comprobad si los resultados obtenidos son los indicados:

m	n	MCD resultante	¿Cuántas veces se ejecuta el bucle?
8	16	8	
40	16	8	
13	120	1	
165	616	11	

Tanto si el programa funciona correctamente o no, deberéis utilizar el depurador para seguir la ejecución de vuestro programa paso a paso. Seguid leyendo para saber cómo debéis depurar.

4.2 A vueltas con el depurador

A la vista del código que habéis implementado, decidid en que línea(s) deberíais colocar el(los) punto(s) de parada de ejecución para ver cómo los valores de las variables más relevantes van cambiando.

Una vez hecho eso cread el(los) breakpoint(s) y lanzad ejecuciones asignando a `m` y a `n` los valores indicados en la tabla que sigue. Completadla utilizando el depurador. Mostrad la tabla a la persona que imparta la sesión de laboratorio.

m	n	Número de iteraciones del bucle	Lista de restos sucesivos	Número de línea donde habéis depositado el <i>breakpoint</i>
8	16			
40	16			
13	120			
165	616			
64	1024			
443	701			

4.3 Implementación en una función independiente del algoritmo de Euclides

La siguiente tarea será crear una función que calcule el mcd de dos números naturales pasados como argumentos.

Cread primero un nuevo package y llamadlo “**edu.upc.etsetb.poo.laboratorio.sesion1.apartado3**”.

Cread en él la clase **MaxComDivisorFuncionIterativo**.

Copiad en ella el código de la clase **MaxComDivisorMainIterativo**.

A continuación, modificad dicho código siguiendo los pasos que se muestran a continuación:

- i. Insertad después de la línea que da fin a la función main() el siguiente código:

```
public int mcd(int dividendo, int divisor){  
}
```

Como veis la primera línea coincide con la cabecera de una función tal y como se escribiría para un programa C, excepto por la palabra reservada public, que os explicarán en las sesiones de teoría.

- ii. Insertad ahora entre la llave de comienzo y la llave de final de la función el código que habéis desarrollado en el main para calcular el mcd de m y n, sin olvidar las sentencias que declaran aquellas variables que no son ni n ni m. Añadid la instrucción que devuelve el resultado (con return, como en una función de C).
- iii. Borrad todas las líneas del main excepto aquellas que declaran y asignan valores a m y a n y la línea que presenta el resultado por consola (System.out.println....).
- iv. Insertad antes de esta última instrucción el siguiente código:

```
int res ;  
  
MaxComDivisorFuncionIterativo obj = new MaxComDivisorFuncionIterativo() ;  
  
res = obj.mcd(m,n) ;
```

No os preocupéis si no entendéis qué hace la segunda línea. Eso es objeto de presentación en las sesiones de teoría.

La tercera instrucción invoca a la función mcd que acabáis de crear (de nuevo, los detalles se os darán en las sesiones de teoría).

- v. Modificad la sentencia System.out.println.... para que presente el valor de la variable res

Una vez realizados estos cambios de forma correcta, deberíais ser capaces de obtener los mismos resultados que antes.

Debéis realizar al menos una sesión de depuración depositando un breakpoint en la línea

```
res = obj.mcd(m,n) ;
```

Cuando el programa se detenga pulsad F7 (Step Intro): observaréis que el programa avanza pero se detiene en la primera instrucción de la función mcd(...) que ejecute alguna acción. A partir de ahí podéis seguir paso a paso una ejecución y comprobar que los resultados parciales son los mismos que en la solución sin función mcd(...)

4.4 Implementación recursiva del algoritmo de Euclides

En la mayoría de los lenguajes de programación moderna, una línea de una función determinada puede invocar a dicha función. A este tipo de funciones se las denomina funciones recursivas. Si una función se invoca a sí misma se dice que se realiza una invocación recursiva directa. Si una función f1 invoca a otra f2 y ésta invoca de nuevo a f1, se dice que en esta secuencia de llamadas se da recursividad indirecta.

Estudiad las notas sobre recursividad que el profesorado de la asignatura ha publicado para entender y aprender qué es la recursividad, cómo debe programarse y cuándo debe utilizarse.

En este apartado escribiréis un programa que incluya una función a la que se le pasen dos números naturales m y n y que devuelva su máximo común divisor.

En muchas ocasiones el código de una función recursiva acaba siendo más sencillo que el código de una función que realiza la misma tarea iterativamente.

Cread primero un nuevo package y llamadlo “**edu.upc.etsetb.poo.laboratorio.sesion1.apartado4**”.

Cread en él la clase **MaxComDivisorFuncionRecursivo**.

Copiad en él el código de la clase **MaxComDivisorFuncionIterativo**.

Borrad el contenido de la función mcd(), es decir, dejad solamente su cabecera y la llave que cierra dicha definición de la función.

Insertad el código recursivo para el cálculo del máximo común divisor en el cuerpo de la función. Para ello implementad el algoritmo que se indica a continuación.

Para diseñar el algoritmo recursivo basta programar la definición recursiva del propio máximo común divisor, que es como sigue:

$$\text{mcd}(\text{dividendo}, \text{divisor}) = \begin{cases} \text{dividendo}, & \text{si divisor es 0} \\ \text{mcd}(\text{divisor}, \text{dividendo} \% \text{divisor}) \end{cases}$$

En consecuencia el algoritmo recursivo para la función `mcd(...)` que calcula el máximo común divisor de dos enteros debe hacer lo siguiente:

- Guardar en la variable `dividendo` el mayor de los dos argumentos y en la variable `divisor` el menor
- Devolver el `dividendo` si el `divisor` es 0.
- Devolver el resultado de invocar a `mcd(divisor, dividendo%divisor)` en caso contrario.

4.5 Depuración de función RECURSIVA

Poned un punto de parada en cada una de las instrucciones que incluyen el `return`.

Cumplimentad una tabla como la que sigue para los siguientes pares de valores

Para `m=8, n=16`.

Para `m=16, n= 40`.

Para `m=616, n=165`.

Número de invocación a <code>mcd(...)</code>	dividendo	divisor	resto(segundo argumento de invocación recursiva)	Línea que devuelve el valor